CEMA Track Write-up

CEMA-H1: Quiet Power

1. What I tried first, and why it did not work

My first thought was to brute-force the 16-byte AES key directly, but 2^128 combinations is far too many to try. I knew the chip's power draw leaked information about what it was doing, but looking through the raw power traces by hand went nowhere, because the noise hid any obvious bit patterns.

2. The insight that actually cracked it

I did not need to guess the whole key at once. Correlation Power Analysis (CPA) lets me guess one key byte at a time. I wrote a script that simulates the AES S-Box output for all 256 possible values of a single key byte and computes the Hamming Weight (the number of 1 bits) for each guess. I then ran a Pearson correlation between those simulated weights and the real power traces. The correct guess showed a clear spike above the noise. I repeated this for all 16 bytes.

3. How I satisfied myself the answer was right

For each byte, the wrong guesses had correlation near zero, and exactly one guess stood out with a high value. I took the 16 best bytes, used them as the AES key, and decrypted the provided payload. It came out as the readable "Intern_Pro_Max" flag, which confirmed the key.

4. Where I used an AI assistant and what I had to correct

I had not heard of Correlation Power Analysis before this challenge. I asked an AI to explain how power analysis works on hardware, and it taught me about comparing Hamming weights using Pearson correlation. Once I understood the theory, I asked it to help write the vectorized NumPy code for the cross-covariance, which made my script much faster. I only had to change the variable names to match my datasets.

CEMA-H2: Mission Seal II

1. What I tried first, and why it did not work

My first idea was a loop that brute-forces the 64-bit squadron key by running the seal function for each guess. I worked out that even at a billion checks per second this would take almost 600 years, so brute force was not an option.

2. The insight that actually cracked it

I read through the provided Python seal algorithm and noticed it only used bit shifts, XORs and Galois Field multiplication. It had no non-linear operation such as an S-Box, so the whole function was linear over GF(2). That meant I could model it as a 64x64 binary matrix. I had 5 missions with 24 bits of the seal exposed for each, which gave me 120 linear equations. I solved them with standard Gaussian Elimination over GF(2) to get the 64-bit key.

3. How I satisfied myself the answer was right

The matrix method is exact, so there was no guessing involved. To confirm it anyway, I put the recovered key back into the original seal algorithm and ran it on all 5 intercepted missions. It reproduced all 5 seals exactly as they appeared in the log.

4. Where I used an AI assistant and what I had to correct

I knew very little about finite field math. I asked an AI how to reverse the seal function, and it explained that because the function only uses XOR and bit shifts, it is linear over Galois Field 2 and can be solved with matrix methods. I had never written a Gaussian Elimination solver for GF(2), so I had the AI write the solve_linear_system_gf2 function. I did not need to change the math. I just fed my generated equations into it.

CEMA-H3: Core Dump 2

1. What I tried first, and why it did not work

My first thought was to search the RAM dump for the 16-byte Master Key, in case it was sitting in plain text near the AES buffers. That would have been a waste of time, because the prompt said a security scrub had already zeroed the Master Key array.

2. The insight that actually cracked it

The challenge hinted that memory gets reused and that other data is left behind. AES expands the Master Key into a larger Key Schedule, so even with the Master Key zeroed, parts of that schedule might still be in RAM. I wrote a scanner that looks for a property of the AES key schedule: w[k] = w[k-1] ^ w[k-4]. It found a 20-byte block that satisfied this XOR rule. With an intact round key in hand, I wrote a script to run the key expansion backward and recover the original Master Key.

3. How I satisfied myself the answer was right

To check the Master Key, I searched the memory dump for the "MSNCRYPT" header to locate the ciphertext and IV. I plugged the recovered key and the IV into a standard AES-CBC decryptor, and it decrypted the payload into the "Intern_Pro_Max" flag.

4. Where I used an AI assistant and what I had to correct

I did not know the internals of AES well, so I asked an AI how the Key Schedule works. It explained the w[k] = w[k-1] ^ w[k-4] relationship. Reversing the key schedule means undoing the Rcon XORs and SubWord steps, which I was not familiar with, so I had the AI write the inverse logic for the get_prev_round_key function. I wrote the memory scanning code myself, and the reversal function worked as given once I passed the block into it.